SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science & Engineering

C02 - ASSESSMENT TOOL - 1

Scenario-Based Assessment

Course Code	**CSA3717**	**Course Title**	**Software Testing**
CO Assessed	**CO2**	**Assessment**	**Assessment Tool 1:**
Weightage	**50% of CIA**	**Total Marks**	**100**
CO2 Statement	*Design effective test cases using black-box, white-box testing techniques and tocontrol methods flow.*		
Student Name	**A.V.Rama krishna reddy**	**Reg. No.**	**192311123**
Section/Batch	**Slot A**	**Date**	**02-11-2026**

Code of Conduct:

I **A. V.Rama krishna reddy**, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Rama krishna

2. A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.

Answer

Introduction

White box testing is a software testing technique in which the internal structure, program code, logic, conditions, and execution paths of an application are examined.
In this scenario, a healthcare application has experienced critical defects after deployment because the testing team did not provide sufficient test coverage.
As a test engineer, the main objective is to identify the missing test cases, examine the internal control flow, and design an effective testing strategy to detect defects before deployment.
A healthcare application must work correctly because it handles important operations such as:

- Patient registration and login.
- Doctor appointment booking.
- Patient medical records.
- Prescription management.
- Payment processing.
- Medical report generation.

For example, if a patient books an appointment but the application fails to save the appointment details, the patient may miss an important medical consultation. Therefore, proper testing is essential.

Evaluation of the Existing Testing Approach

The critical defects found after deployment indicate that the previous testing process was incomplete.
The following problems may have occurred:

1. Insufficient statement coverage
Some statements in the application code were not executed during testing. This means certain operations may contain undetected errors.

2. Incomplete branch coverage
The testing team may have tested only successful conditions and ignored failure conditions.
For example:

- Valid patient details.
- Invalid patient details.

Both conditions must be tested.

3. Missing execution paths
Some combinations of conditions were not tested, such as valid patient details with an unavailable doctor.

4. Poor error-handling testing
The application may not have been tested for database failures, invalid inputs, empty fields, or incorrect appointment dates.

5. Lack of regression testing
After correcting defects, the team may not have retested the previously working functions. This can introduce new defects.

6. Inadequate integration testing
Patient registration, appointment booking, doctor availability, and payment modules may not have been tested together.

Application of White Box Testing Techniques

White box testing examines the internal implementation of the software. The following techniques can be applied to the healthcare application.

A. Statement Coverage

Statement coverage ensures that every executable statement in the program is executed at least once.

Example:

Consider the following appointment booking logic:

```
def book_appointment(patient, doctor):
    if patient == "":
        print("Invalid patient")
    else:
        if doctor == "available":
            print("Appointment confirmed")
        else:
            print("Doctor unavailable")
```

Test cases:

- Test Case 1: Patient is empty.
- Test Case 2: Patient is valid and doctor is available.
- Test Case 3: Patient is valid and doctor is unavailable.

These test cases execute all the statements in the program.

Formula:

B. Branch Coverage

Branch coverage ensures that every decision outcome is executed at least once.

In the healthcare application, decisions include:

- Is the patient information valid?
- Is the doctor available?
- Is the appointment date valid?
- Is the payment successful?

For example:

Condition	**True Branch**	**False Branch**
Patient details valid?	Continue booking	Display error
Doctor available?	Confirm appointment	Show unavailable
Payment successful?	Generate receipt	Display payment failure

Both True and False branches must be tested.

Formula:

C. Path Coverage

Path coverage verifies different possible execution paths through the program.
For a healthcare appointment system, the following paths should be tested:

1. Valid patient details → Doctor available → Appointment confirmed.
2. Invalid patient details → Error message.
3. Valid patient details → Doctor unavailable → Unavailable message.
4. Valid patient details → Doctor available → Payment failure.
5. Valid patient details → Invalid appointment date → Date validation error.

Path testing helps identify errors that may not be detected through statement coverage alone.

D. Condition Coverage

Condition coverage checks whether each individual condition evaluates to both True and False.
For example:

```
if age >= 18 and phone != "":
   print("Registration successful")
else:
   print("Invalid details")
```

Test cases:

- Age = 20, phone number entered → True.
- Age = 16, phone number entered → False.
- Age = 20, phone number empty → False.

This helps verify individual conditions in the program.

Construction of a Conceptual Control Flow Graph (CFG)

A Control Flow Graph (CFG) is a graphical representation of the execution flow of a program.
It contains:

- **Nodes:** Individual statements or groups of statements.
- **Edges:** Flow of control between nodes.
- **Decision nodes:** Conditions that produce True or False outcomes.
- **Entry node:** Starting point of the program.
- **Exit node:** Ending point of the program.

Consider a healthcare application that allows patients to book doctor appointments.
Execution steps:

1. Start the application.
2. Enter patient details.
3. Validate patient information.
4. Check whether patient details are valid.
5. Check doctor availability.
6. Confirm appointment or display an error.
7. End the process.

Control Flow Graph Diagram

Node 1: Start
Node 2: Enter Patient Details
Node 3: Validate Patient Details
Node 4: Patient Details Valid?
YES
Node 5: Check Doctor Availability
Node 6: Slot Available?
YES
Node 7: Confirm Appointment
NO
Node 8: Display Invalid Details
NO
Node 9: Display Slot Unavailable
Node 10: End

The CFG helps the testing team understand how the program moves from one statement to another and identify untested decision branches.

Identification of Missing Execution Paths

After examining the control flow graph, the following execution paths should be considered.

Path	Execution Flow	Expected Result
Path 1	Start → Enter details → Validate → Valid → Check slot → Available → Confirm	Appointment confirmed
Path 2	Start → Enter details → Validate → Invalid	Error message
Path 3	Start → Enter details → Validate → Valid → Check slot → Unavailable	Slot unavailable
Path 4	Start → Enter details → Validate → Valid → Check slot → Available → Confirm	Appointment saved successfully

Additional exceptional paths must also be tested:

- Database connection failure.
- Duplicate appointment booking.
- Invalid date and past appointment date.
- Missing patient information.
- Network failure during confirmation.
- Two patients attempting to book the same doctor slot simultaneously.

Why are missing paths important?
If the testing team checks only the successful appointment path, errors in invalid-input handling and doctor availability checking may remain undetected. Testing all meaningful paths helps prevent such failures.

Design of Effective Test Cases

The following test cases are designed using white box testing and control flow methods.

Test Case ID	Test Scenario	Input	Expected Output
TC01	Valid patient registration	Correct patient details	Registration successful
TC02	Invalid patient details	Empty name	Error message
TC03	Valid appointment booking	Valid details and available doctor	Appointment confirmed
TC04	Doctor unavailable	Valid details and unavailable doctor	Slot unavailable
TC05	Invalid appointment date	Past date	Date validation error
TC06	Missing phone number	Empty phone number	Validation error
TC07	Database failure	Database disconnected	System error message
TC08	Duplicate appointment	Same patient and slot	Duplicate booking prevented
TC09	Payment failure	Invalid payment or declined transaction	Payment failure message
TC10	Network interruption	Network disconnected during booking	Safe failure and recovery

These test cases cover normal operations, invalid conditions, decision branches, and exceptional situations.

Evaluation of Test Adequacy Criteria

Test adequacy criteria determine whether the designed test cases are sufficient to test the software.
The main criteria are:

1. Statement Coverage
Checks whether every executable statement has been executed at least once.

2. Branch Coverage
Checks whether all decision outcomes, including True and False, have been executed.

3. Condition Coverage
Checks whether each individual condition has produced both True and False results.

4. Path Coverage
Checks whether all important execution paths have been tested.

5. Requirement Coverage

Ensures that all healthcare application requirements are covered by test cases.

Comparison of Coverage Techniques

Coverage	Main Purpose	Importance
Statement	Execute all statements	Detects unexecuted code
Branch	Test all decision outcomes	Detects decision errors
Condition	Test individual conditions	Detects logical errors
Path	Test execution sequences	Detects path-related defects

Critical assessment:

Statement coverage alone is not sufficient because executing every statement does not guarantee that every decision outcome is tested. Branch and condition coverage provide stronger checks. Path coverage gives additional confidence, but complete path coverage may be impossible for complex programs containing loops and many combinations.

Therefore, a combination of these techniques should be used rather than depending on only one coverage criterion.

Improved Testing Strategy

To avoid critical defects after deployment, the healthcare application should follow a structured testing strategy.

Step 1: Requirement Analysis

Understand all healthcare application requirements, including registration, login, appointments, payments, and medical records.

Step 2: Code Analysis

Examine the internal program structure and identify statements, decisions, conditions, and loops.

Step 3: Control Flow Graph Construction

Create a control flow graph to identify different execution paths and decision points.

Step 4: Test Case Design

Prepare test cases for valid inputs, invalid inputs, boundary conditions, and exceptional situations.

Step 5: Statement and Branch Coverage

Execute all statements and verify both True and False decision outcomes.

Step 6: Condition and Path Testing

Test individual logical conditions and all important execution paths.

Step 7: Integration Testing

Verify the interaction between patient registration, doctor availability, appointment booking, payment, and database modules.

Step 8: Regression Testing

After fixing defects, execute previously passed test cases to ensure that existing functions are not affected.

Step 9: Final Validation

How the Improved Approach Enhances Reliability and Reduces Risk

The improved white box testing strategy provides several benefits:

1. **Early defect detection**: Identifies coding errors before deployment.
2. **Improved reliability**: Ensures that important healthcare functions work correctly.
3. **Better decision testing**: Verifies all important True and False conditions.
4. **Reduced operational risk**: Prevents appointment booking failures and incorrect patient information processing.
5. **Data accuracy**: Helps ensure that patient records are stored and retrieved correctly.
6. **Improved security**: Helps identify weaknesses in authentication and access-control logic.
7. **Better maintenance**: Regression testing ensures that fixing one defect does not create another.
8. **Patient satisfaction**: Reduces application failures and improves the overall healthcare experience

Real-Time Examples

Consider a patient named Rahul who wants to book an appointment with a cardiologist.

- Rahul enters valid patient details.
- The application checks his information.
- The application checks the cardiologist's available slots.
- If a slot is available, the appointment is confirmed.
- If no slot is available, the application displays an unavailable message.
- If Rahul enters incorrect details, the application displays a validation error.
- If the database connection fails, the application displays an error instead of confirming an unsaved appointment.

Conclusion

- White box testing and control flow testing are important techniques for improving the quality of a healthcare application. By applying statement coverage, branch coverage, condition coverage, and path coverage, the testing team can identify missing execution paths and internal programming errors.
- Black box testing can additionally verify the application's functionality using valid, invalid, and boundary inputs without examining its internal code.
- For example, patient registration, appointment booking, doctor availability, and payment processing can be tested using black box techniques, while the internal conditions and execution paths can be verified using white box techniques.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements.	Clearly understands the scenario with minor gaps.	Basic understanding; some aspects of the scenario are unclear.	Poor understanding or misinterpretation of the scenario.
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems.	Applies relevant concepts correctly with minor errors.	Applies basic concepts but with limited accuracy.	Fails to apply appropriate concepts.
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification.	Demonstrates good analysis and selects reasonable solutions.	Limited analysis; solutions lack justification.	Poor or no analysis; incorrect decisions.
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach.	Logical and structured solution with minor gaps.	Basic solution with limited structure or feasibility.	Unclear or incorrect solution approach.
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning.	Clear explanation with some justification.	Limited clarity and weak justification.	Poorly explained with no justification.
Total	100				